Proceedings of the 20th Linux Audio Conference (LAC-26), Maynooth, Ireland, June 18–20, 2026

SEAMLESS: DISTRIBUTED SPATIAL AUDIO RENDERING ON THE LINUX AUDIO STACK

Fares Schulz

Technische Universität Berlin (TUB)
Berlin, Germany
fares.schulz@tu-berlin.de

Maximilian Weidauer

Technische Universität Berlin (TUB)
Berlin, Germany
weidauer@campus.tu-berlin.de

Stefan Weinzierl

Technische Universität Berlin (TUB)
Berlin, Germany
stefan.weinzierl@tu-berlin.de

Henrik von Coler

Georgia Tech
Atlanta, GA, USA
hvc@gatech.edu

ABSTRACT

Loudspeaker-based spatial audio methods, such as Ambisonics and Wave Field Synthesis (WFS), benefit from large channel counts, but the computational and organizational demands of driving hundreds of speakers exceed the capabilities of a single machine. Despite the existence of various rendering systems, there is still a lack of open-source solutions that distribute processing across multiple hosts while remaining accessible to artists. We present SeamLess, a modular, real-time spatial audio rendering system that distributes processing across multiple Linux servers. Its core components – a flexible multichannel audio processor and an OSC-based message router – separate the rendering backend from user interaction, allowing control through DAW plug-ins or plain OSC messages. Built on the JACK audio server and, more recently, PipeWire, SeamLess has been running continuously in a permanent museum exhibition since May 2025. We report on its architecture, operational experience, and challenges encountered. We argue that SeamLess offers a viable, fully open-source solution for both large-scale venues and smaller, budget-conscious setups. PipeWire's native AES67 support provides a promising foundation for future development.

1. INTRODUCTION

A variety of sound reproduction methods besides two-channel and multi-channel stereo have been developed over the past few decades. Commonly used, non-proprietary methods include Ambisonics and Wave Field Synthesis (WFS).

Ambisonics is a domain-independent audio format that allows spatial audio information to be decoded for arbitrary speaker setups. All virtual sound sources are projected onto a sphere around the listener, and the encoded signal can be played back on any loudspeaker layout using an appropriate decoder. WFS is typically used as an object-based spatialization method, synthesizing wave fronts in a room by overlapping signals from a large number of speakers [1]. This also enables virtual sources to be rendered in a way that makes them sound as though they are positioned within the speaker array, a concept referred to as focused sources [2]. However, WFS is predominantly employed in two-dimensional setups and therefore lacks native support for elevated sound sources.

Combining these methods in a single system can leverage their complementary strengths: the perceptual accuracy and focused-source capability of WFS for nearby, horizontally arranged sources, and the flexibility of Ambisonics to render diffuse or elevated, three-dimensional sound fields.

The SeamLess software system was developed to seamlessly combine different spatial sound reproduction methods. Five years after the release of the first version [3], the system has been refined and is in use in several installations, including the Listening Room of the permanent exhibition at the Ethnological Museum in the Humboldt Forum in Berlin [4], the Electronic Studio at TU Berlin (Fig. 1), and the Lecture Hall H 0104 at TU Berlin. The Humboldt Forum's dedicated, acoustically treated listening room combines a 256-channel WFS system arranged across two opposing arcs with a 45-speaker HOA dome spanning three height levels, accommodating up to 20 standing visitors. The TU Studio is equipped with a 192-channel WFS system in an irregular octagonal arrangement, a 21-channel Ambisonics dome, and a classic ring of eight loudspeakers, serving as a versatile facility for music production and research. The lecture Hall H 0104 operates over 800 WFS channels and seats an audience of more than 600 people.

Figure 1: *SeamLess system in the listening room of the Ethnological Museum at the Humboldt Forum in Berlin (top) and in the electronic studio of the Audio Communication Department at TU Berlin (bottom)*

To support these diverse venues, the SeamLess system is built around a modular, flexible software architecture that integrates different spatial sound reproduction methods to be integrated into a distributed, server-based rendering backend. Users can control the system via audio streams and Open Sound Control (OSC) messages. Currently, the system supports WFS and

Ambisonics rendering, as well as a dedicated LFE channel. However, system is designed to be easily extensible to additional methods in the future. The software stack is Linux-first, fully free and open source, except for the Dante audio drivers and the REAPER-based playback solution.

For maximum modularity, the SeamLess software stack consists of several interchangeable components, each with a specific role in the system. The two main components are the Audio Matrix, responsible for audio processing and routing, and the OSC-Kreuz, responsible for distributing OSC messages. Current rendering engines include the tWonder WFS renderer and a modified version of the IEM Ambisonics AllRA decoder. Both engines receive audio and control data via the Audio Matrix and OSC-Kreuz. Users can control spatial rendering via the SeamLess Plug-in Suite, a set of plug-ins for digital audio workstations (DAWs) that send OSC messages to the OSC-Kreuz or other OSC-capable software or hardware. The Showcontrol software is used for scheduling the daily playback, which is necessary in a permanent installation context. Audio routing between the different components is handled using the Jack Audio Connection Kit (JACK)[1] or PipeWire[2]. The connections are automatically set by the Jack Connection Manager, a custom software component developed for this purpose.

To orchestrate this complex system, we use Ansible for distributed deployment and provisioning while all components follow a declarative configuration approach based on YAML files. Strict version control is enforced across all components and multiple versions can coexist simultaneously, enabling straightforward testing and rollback when necessary. Systemd services manage the startup and shutdown of each component, ensuring robust and reliable operation in a fully headless server environment. Reliability has been a central concern throughout development, given that one of the primary use cases involves the daily playback of works in a museum context. In this scenario, the system must run uninterrupted for several consecutive days, a requirement that the software stack can meet.

In addition to operational robustness, the server-based architecture offers a fundamental advantage for creative workflows. As spatial audio rendering systems grow in complexity, client-side approaches require users to invest considerable effort in configuring and adapting the system to each loudspeaker arrangement. By offloading the rendering to a dedicated cluster, location-specific configuration such as loudspeaker positions, decoder matrices, and rendering engine parameters, only need to be defined once during installation. Users can then play back exactly the same material on different sites without modification, which greatly simplifies the transfer of content between venues. The cluster-based approach also eliminates the computational limitations of a single workstation, enabling the operation of large-scale systems. The largest SeamLess installation to date comprises six Linux servers driving an 832-channel loudspeaker setup in Lecture Hall H 104.

These features make SeamLess a viable solution for large-scale loudspeaker setups, as it provides an easily deployable rendering backend on standard hardware that is accessible to artists with limited technical knowledge and adaptable to a variety of spatial sound reproduction methods. Furthermore, the project aims to promote the use of the PipeWire audio server in environments with a large number of channels. To our knowledge, SeamLess is the first system to employ PipeWire on this scale. We believe that its integration with the WirePlumber[3] session manager and its native support for the AES67 networked audio protocol make PipeWire a particularly promising foundation for future spatial audio systems.

[1] https://jackaudio.org/

[2] https://pipewire.org/

[3] https://pipewire.pages.freedesktop.org/wireplumber

2. RELATED WORK

Several proprietary and open-source software systems for spatial audio rendering have been developed, each with a different scope and target audience. They can be broadly grouped into integrated rendering frameworks that process audio themselves, and plug-in suites that operate within a DAW host. We review representative examples of both categories before discussing how SeamLess differs from the existing landscape.

2.1. Integrated Rendering Frameworks

Among the most comprehensive integrated systems is IRCAM's Spat [5]. It supports a wide range of panning algorithms, including Vector-Based Amplitude Panning (VBAP), Ambisonics, Distance-Based Amplitude Panning (DBAP) and Wave Field Synthesis (WFS), as well as perceptual room acoustics simulation. Originally a collection of Max/MSP[4] externals, it has since been refactored into a standalone C++ library with bindings for Max, audio plug-ins, OpenMusic and Matlab. Nevertheless, Max remains the primary user-facing environment, creating a dependency on a commercial host. All processing runs on a single machine, and Spat only targets macOS and Windows, with no official Linux support. Furthermore, Spat is free, but under a proprietary license.

In terms of hardware, Holoplot[5]'s WFS panels have brought about the biggest leap in WFS technology in recent times, notably employed in the Las Vegas Sphere. Their custom rendering software runs on an embedded system built into the speaker. However, this software is also proprietary and only functions with Holoplot's custom hardware.

Regarding open-source alternatives, the SoundScape Renderer (SSR) [6, 7] is a C++ framework for real-time spatial audio rendering built on the JACK audio server. SSR is available for Linux and macOS and is licensed under the GPLv3. It supports VBAP, Ambisonics, and WFS and it provides an extensible architecture for custom rendering algorithms, although new methods must be implemented in C++ and compiled against the framework. While the graphical front end can run on a separate machine, all audio rendering is confined to a single host.

With a narrower focus, TASCAR [8, 9] is a Linux-native, GPLv2-licensed, JACK-based application primarily developed for hearing aid research. It focuses on dynamic acoustic scenes with moving sources, Doppler simulation, and motion tracking integration, and provides command-line interfaces for Matlab and Octave. However, it is oriented toward controlled experimental scenarios rather than large-scale loudspeaker installations.

2.2. DAW Plug-in Suites

A complementary approach to spatial audio is provided by plug-in suites that run inside a DAW host.

The IEM Plug-in Suite [10] and the Ambisonic Plug-in Suite by Kronlachner [11] both center on Ambisonics-based workflows within a DAW. The IEM suite provides a comprehensive collection of VST plug-ins, including encoders, a flexible AllRAD decoder, and room simulation tools. It is widely used in both research and production. Kronlachner's suite takes a similar approach by offering encoding, decoding, and manipulation of Ambisonic signals at variable spatial resolutions. It has been successfully deployed in performances with hemispherical and spherical loudspeaker setups of varying size. Both suites are built with the JUCE[6] framework and can therefore also run as standalone JACK applications on Linux. Neither suite, however, supports WFS or headless distributed operation, which keeps them confined to single-machine rendering.

[4] https://cycling74.com/products/max

[5] https://www.holoplot.com/

[6] https://juce.com/

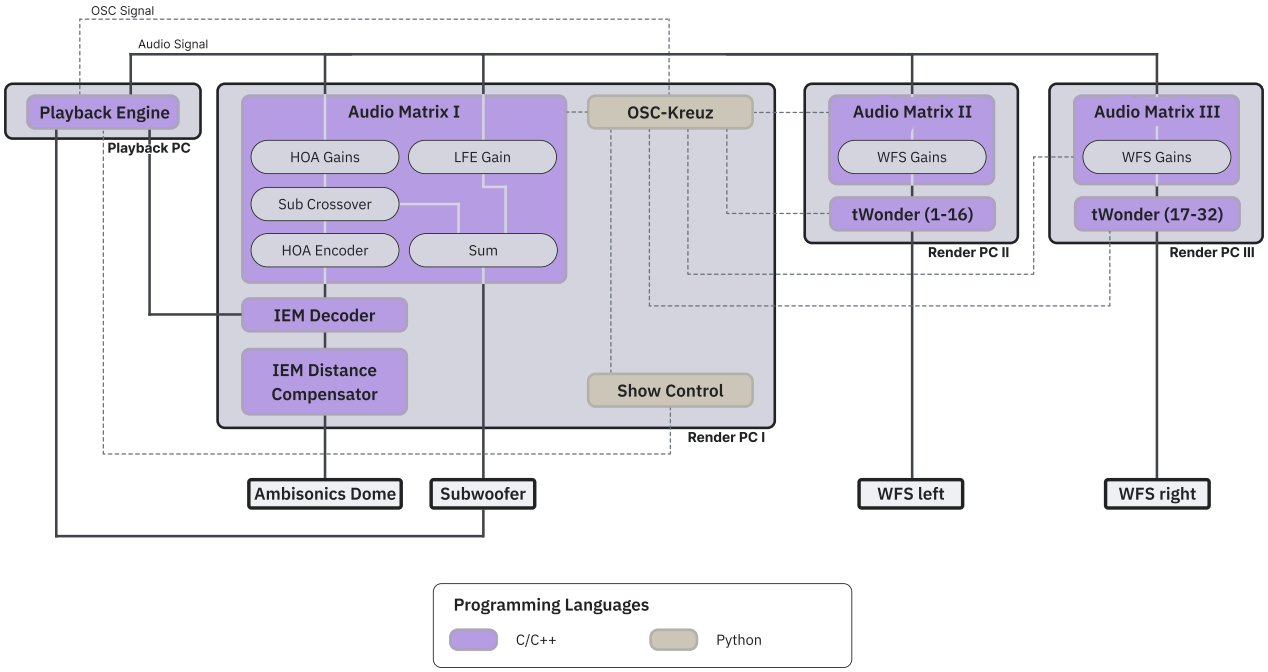

Figure 2: *Signal flow of the SeamLess system in the Humboldt Forum*

SPARTA [12] is an open-source plug-in suite built on the Spatial Audio Framework (SAF) and JUCE. It covers the full Ambisonics workflow up to 10th order, including microphone array encoding, loudspeaker decoding, and headphone decoding. It places a strong emphasis on binaural reproduction with personalized HRTFs via SOFA files and extends to 6DoF rendering. As an exclusive VST, VST3, AU, LV2 and AAX plug-in suite, it requires a DAW host and is oriented toward studio production and research, but does not support WFS.

2.3. Limitations of Existing Systems

As the above systems were developed primarily as single-machine applications, features such as deployment automation and complex show scheduling were not central to their design. More critically, none of them are designed around an architecture that distributes rendering across multiple networked machines to power large loudspeaker arrays. SeamLess addresses this gap by using a dedicated backend cluster for all computationally intensive rendering while users interact with the system through lightweight clients, such as DAW plug-ins or plain OSC messages. Distributed, multi-machine rendering is therefore the primary architectural driver. Along with its clear separation of concerns between the rendering backend and user-facing interaction, headless server operation, full Linux support, open-source licensing, deployment automation, and show scheduling for permanent installations, these characteristics distinguish SeamLess from existing systems.

3. IMPLEMENTATION

The SeamLess system uses a modular approach spread over multiple Linux machines. Figure 2 shows the signal flow for the Listening Room in the Humboldt Forum Berlin. All spatialization and processing of audio is performed on rendering machines, while the playback machine only sends out mono audio streams, intended as virtual sources, encoded Ambisonics audio streams, or LFE signals via Dante or MADI, as well as positional data via OSC. Synchronization of the audio streams between the machines is managed by the audio hardware, depending on the system, either using Dante or using an external word clock. For synchronization to be correct, all rendering systems must operate on the same buffer sizes.

The audio of the playback machine is routed to all rendering machines, where it is then further processed and fed into the rendering software. On the rendering machines, the audio is routed using either JACK or PipeWire with Pipewire-Jack as the backend. This enables connections between different processing applications. The switch to PipeWire was only recently made possible by removing the previous 64-channel hardware limit.[7]

The OSC control messages are sent from the playback machine to the OSC-Kreuz running on one of the renderers via the network. The OSC-Kreuz then supplies all receivers with the necessary data. All components start as systemd user services, which enables easy dependency management and startup checking using systemd notify, as well as simple restarts. The core components will be elaborated on further in the following sections.

All components are configurable in a flexible manner to support different usage scenarios, usually via YAML config files. The base coordinate system of the playback-system is laid out as shown in Figure 3. The coordinate system is normalized so that the wall farthest from the center is at a distance of 0.5 on either the x- or y-axis. This enables transfer between different locations using the SeamLess system without having to rescale all positions. Due to the system's modular design, new components can easily be added, even during runtime. Custom visualizations of the system can subscribe to the OSC data streams, and alternative renderers can replace the current ones, provided they communicate using OSC.

[7] The limit was raised in PipeWire commit c91f75ae2e68e601e5f90790704776cd1b3f755c, with additional fixes being implemented in subsequent months.

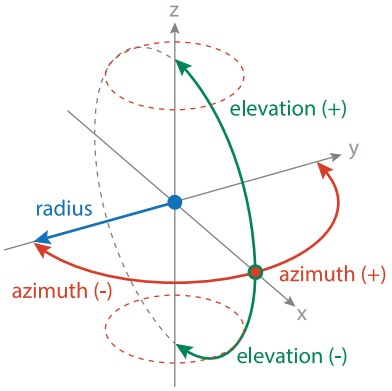

Figure 3: *The coordinate system used by the SeamLess system, with the x-axis pointing into the view direction of a listener positioned in the center of the room.*

3.1. Audio Matrix

The Audio Matrix[8] is a flexible, multichannel DSP program written in C++. An exemplary signal flow graph is shown in Figure 4. It takes a configurable number of inputs, which are then routed into the individual tracks. Tracks can be understood as the preprocessing for the different output systems, e.g. WFS or Higher Order Ambisonics. Each track contains multiple modules that perform the actual processing. Many modules can be controlled via OSC to set gains or change the position of an Ambisonics-encoded source. During initialization, each module on each track is informed of the channel count of the previous module. This ensures that the correct number of buffers are initialized. This also ensures the correct number of output channels for a given processing chain.

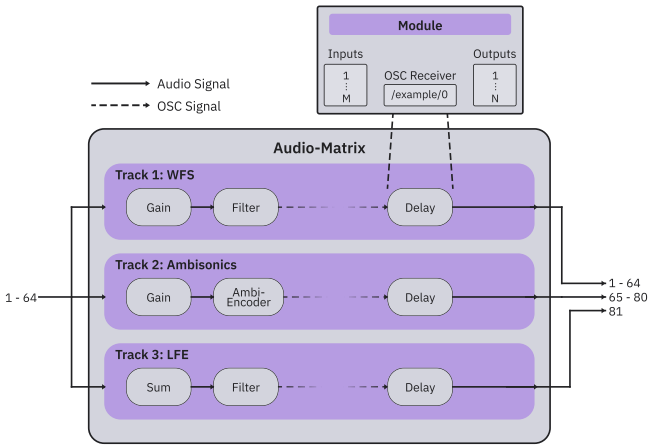

Figure 4: *Signal flow within the Audio Matrix*

The Audio Matrix and its tracks are configured via a YAML config file. An example config can be seen in Section 8.1. The currently implemented modules are the following:

- **Gain**: Applies individual gains controllable via OSC to each channel.
- **Ambisonics Encoder**: A simple Ambisonics encoder that supports up to 4th order Ambisonics. Each incoming channel has its own position, which is controllable via OSC.
- **Sum**: Sums all incoming channels up into one output channel.
- **Filter**: Linkwitz-Riley low- or highpass filters are used as crossover filters.
- **Distance Gain**: Adjusts the gain of a channel based on its distance from the center. The same gain function is used in the Ambisonics Encoder to emulate distance.
- **Delay**: A simple, non-interpolating delay line to adjust for latencies between different playback systems.

Additional modules can be added with little effort.

3.2. OSC-Kreuz

The OSC-Kreuz[9] serves as a central interface for OSC messages in a multi-server spatial rendering environment. It automatically translates incoming OSC messages to match the rendering engines' expected formats and distributes them to all connected receivers. It is written in Python. Receivers are specified in a config file; alternatively, they can subscribe during runtime using an OSC endpoint. Most SeamLess components have specialized receiver classes. However, due to the extensive configurability of the Audio Matrix and its OSC paths, the related receiver class in the OSC-Kreuz is also highly configurable, enabling the use of many different OSC-powered programs. If a receiver requires more specialized OSC messages, a new receiver subclass can be created for it. The OSC-Kreuz also optionally rate-limits the receivers per source. Among other benefits, this leads to smoother movement in the WFS renderers.

An example receiver config is shown in Section 8.3. In this setup only the Audio Matrix receivers are specified explicitly, the tWonder instances and the Showcontrol source viewer subscribe to the OSC-Kreuz during runtime. For the Audio Matrix receiver type, multiple hostname/port pairs and OSC paths can be specified, as either a gain type of a specific rendering engine or as a position type, where the coordinate format is specified.

3.3. Wonder

Wonder[10] [13, 14] is a WFS rendering suite developed at TU Berlin since 2004 and written in C++. For the current iteration of the SeamLess system, most components, such as the message router cWonder and the GUI xWonder have become obsolete; only the actual WFS rendering component tWonder remains. The OSC-Kreuz was expanded to fulfill the cWonder's role of supplying the necessary control data and information about the room polygon. What sets Wonder apart from other WFS rendering tools is its ability to run distributed on multiple machines, eliminating the channel limit posed by the number of audio channels on sound cards, as well as the processing power of the machines. To enable distributed rendering, each tWonder instance knows the positions of the speakers it is responsible for, the positions of the virtual sources and the general shape of the full speaker layout. This allows each instance to determine whether to contribute to the rendering of a given virtual source and whether to treat that source as focused.

For focused sources, this global layout knowledge is essential: the illusion of a source located inside the listening area breaks down if a partial wavefront reaches a listener before the wavefronts "originating" at the virtual source position do. To prevent this, only the speakers on one side of the array polygon participate in the rendering of a focused source. For unfocused sources, only the speaker panels behind which the source is positioned are involved in the rendering.

The primary rendering method applies gains and delays to all speakers, based on their distance from the virtual sound source. Multiple tWonder instances are spawned on each machine, each rendering for a subset of speakers. Recently, tWonder was updated to also receive OSC via multicast, making sending more efficient and receiving more consistent. Each tWonder instance starts as a systemd template service.

[8] https://github.com/tu-studio/audio-matrix

[9] https://github.com/tu-studio/osc-kreuz/

[10] https://github.com/tu-studio/wonder

3.4. Ambisonics Decoding

The SeamLess system supports the encoding of incoming mono audio using the Ambisonics Encoder in the Audio Matrix, as well as the decoding of already encoded Ambisonics recordings up to 3rd order, supplied by the user. The *AllRADecoder* from the IEM Plugin Suite[11] [10] is used as Ambisonics decoder in a modified headless version[12]. Additionally the *Distance Compensator* is used to adjust for time delays due to different speaker distances from the room center.

3.5. SeamLess Plugin Suite

The SeamLess Plugin Suite[13] is used on the client side, on the artist's playback PC, in their Digital Audio Workstation (DAW) of choice. It is used to control the positions and gains of the virtual sound sources. Built using the JUCE framework, it and can be used in almost all DAWs. They plugins perform no audio processing on their own; they only send OSC messages to the OSC-Kreuz.

The suite consists of a *SeamLess Main* plugin, that handles incoming and outgoing OSC communication and a *SeamLess Client* plugin that controls and displays positional data and rendering system gains for a single virtual source. Multiple *SeamLess Clients* connect to a single *SeamLess Main* plugin using inter-process communication. The plugins are shown in Figure 6. This split into *SeamLess Main* and *SeamLess Client* plugins enables keeping the positional data (and its automation) grouped with the related audio data.

3.6. Playback Engine

Any program capable of multichannel audio playback and OSC output, or of hosting VST plug-ins, can serve as the front-end for the SeamLess system. Currently REAPER[14] is used as the playback engine, chosen for its stability, OSC-based remote control and Lua scripting capabilities.

3.7. Showcontrol

Showcontrol [15] is a scheduler for managing continuous playback in an exhibition context. Its backend is written in Python, the frontend partially uses Flask, with newer parts using Typescript / React. Playback in REAPER is controlled using OSC messages, with additional functions provided by the REAPER SWS Extensions [16]. Since the Humboldt Forum also employs multiple video screens to add a visual dimension to the pieces played there, the Showcontrol also controls the video players using UDP packets. Pieces, blocks, and the schedule are configured using a set of YAML files. The schedule is generated by reading different blocks of pieces, calculating the pieces' start times, and writing the finished schedule to multiple machine-readable files for actual scheduling, as well as to different summarized files for increased human readability.

Over time, the Showcontrol has developed into a more comprehensive control interface by embedding additional functionality, such as the source viewer – a webGL based visualization of the current source position, as shown in Figure 5 – and by supplying a web API to provide status information about playback and scheduling, as well as to automatically stop playback in case of emergencies. This API is also used for info screens in the museum that display additional information about the piece currently playing and the upcoming schedule.

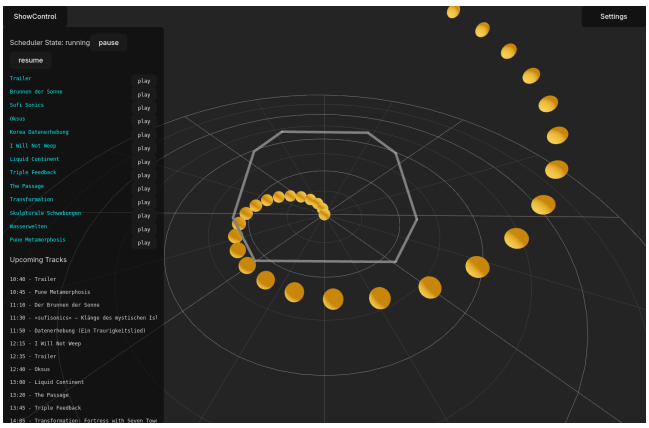

Figure 5: *Screenshot of the Showcontrol source viewer. The yellow spheres represent the position of virtual sources, the grey outline shows the outline of the WFS speaker array, the sidebar on the left enables control over playback and the scheduler*

3.8. Jack-Connection-Manager

The Jack-Connection-Manager[17] is a simple Python application to automatically set JACK connections, built specifically for setups with a high number of connections, while still allowing for a readable configurations. A simple config file could resemble the one in section 8.2. Rather than specifying every single connection, clients are configured and groups of outputs are defined. Existing JACK connection managers aim to create snapshots of the current connections, which is tedious in a setup with many parallel audio channels as needed for real-time object-based spatial audio rendering. The connection manager can run as a daemon and connect all newly connected clients.

If the switch to PipeWire is successful, the Jack-Connection-Manager will likely be replaced by WirePlumber, assuming the machine-specific configuration effort is similar.

3.9. Orchestration

Managing a cluster of rendering machines becomes a lot more efficient and streamlined by automating repetitive tasks like placing and updating config files. All changes and installations on the machines are therefore controlled using the Ansible[18] server automation platform. The Meson[19] build system is mainly used on the machines themselves, even in workflows that are technically not software building.

3.9.1. Configs

The config files for all the components are stored in a separate repository[20]. Using the Meson build system, all configurations are installed into the proper directories. To accomplish this, the build script is supplied with the name of the playback system (*location*) and the identifier for the current machine (*node*). The correct config files are then copied to their directories using this information.

3.9.2. Versioned Install

Due to the reliability requirements of a museum exhibit, the system must be highly stable and have the ability to quickly revert to an older version in case of an issue. To facilitate

[11] https://plugins.iem.at/
[12] https://github.com/TU-Studio/IEMPluginSuite
[13] https://github.com/tu-studio/seamless-plugin-suite
[14] https://www.reaper.fm/
[15] https://github.com/tu-studio/showcontrol
[16] https://sws-extension.org/
[17] https://github.com/tu-studio/jack-connection-manager
[18] https://github.com/ansible/ansible
[19] https://mesonbuild.com/
[20] https://github.com/tu-studio/seamless-configs

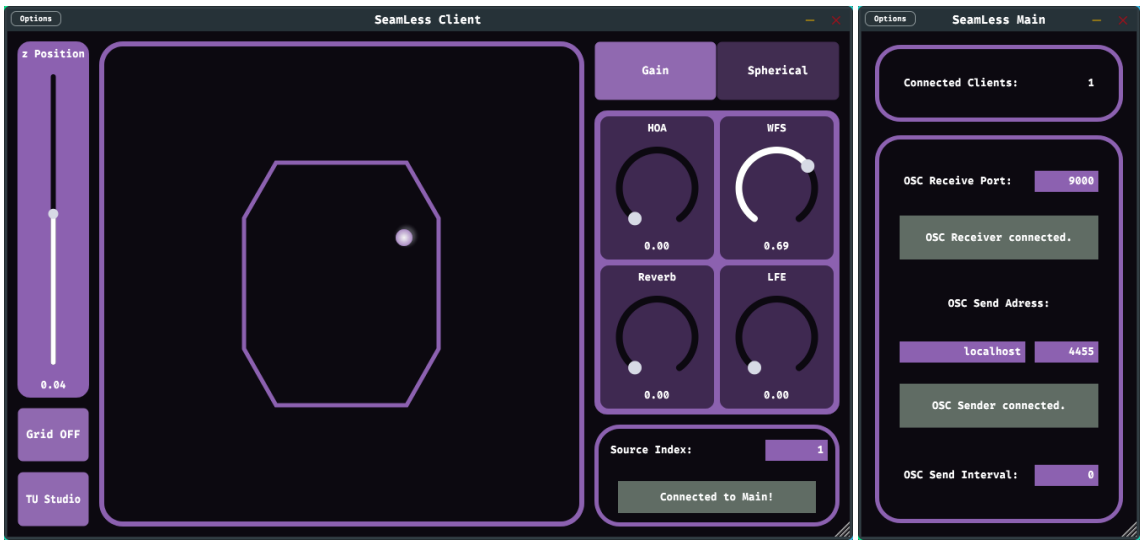

Figure 6: *Screenshot of the SeamLess Client plugin (on the left side) and the SeamLess Main plugin (on the right side)*

this, a string based on the current version (taken from the git tag) is appended to the filename or install directory of all installed programs and files. Instead of the program or directory itself, a symlink to the most recent version is created. Consequently, reverting to an older version simply requires changing the symlink. Meson handles this renaming and symlink workflow (and CMake[21] in the case of the audio-matrix). Some examples of file structures using this concept are shown in the following:

```
/usr/local/bin/
├── audio-matrix -> /usr/local/bin/audio-
     matrix-<version>
└── audio-matrix-<version>
/usr/local/etc/
├── audio-matrix -> /usr/local/etc/audio-
     matrix-<location>-<node>-<version>/
└── audio-matrix-<location>-<node>-<
     version>
```

In the case of a python program the virtual environments are also created in a versioned structure:

```
/usr/local/share/
└── osc-kreuz-<version>/
    └── venv/
/usr/local/bin/
├── osc-kreuz -> /usr/local/bin/osc-kreuz
     -<version>
└── osc-kreuz-<version> -> /usr/local/
     share/osc-kreuz-<version>/venv/bin/
     osc-kreuz
```

3.9.3. Deployment

All machines are managed using Ansible, following the "Infrastructure as Code" principle: a set of playbooks[22] provisions a cluster of fresh Debian installations into a complete SeamLess system in a single run.

Per-room inventory files specify the *service*s running on each machine, the relevant *audio driver*, and the room name, so the playbooks themselves remain untouched when the software stack changes. The inventory variables filter out tasks not required on a given machine and are in part passed to the Meson install scripts, for example to deploy the correct config files. This makes installations reproducible across machines while allowing rapid iteration during testing; auxiliary tasks such as distributing videos to the video players are handled by Ansible as well.

Setting up a new SeamLess system thus requires only three steps: defining an inventory file for the target machines (examples are provided in the playbook repository[22]), creating config files for the desired components (examples are provided in the config repository[20]), and finally running the main playbook, which installs all software and configs on the machines.

4. EXPERIENCE AND CHALLENGES

The SeamLess system was developed through an iterative process involving extensive communication with artists and technical stakeholders. This collaboration was essential in achieving the system's current stable and usable state. Currently, the system reliably starts since there are no longer any boot dependencies between rendering machines. The program has run uninterrupted in the Humboldt Forum since around May 2025, with occasional restarts after updates.

Building our infrastructure on Linux and JACK makes the SeamLess system both flexible and stable. However, the switch to PipeWire offers significant potential, including the built-in scripting capabilities of WirePlumber and the future capabilities of pipewire-aes67 for spatial audio systems. It also improves the setup on machines with multiple sound cards. Due to the recent developments, we will closely monitor the stability of the PipeWire setup, and remain optimistic that it will soon be incorporated into our multichannel and spatial audio setup. REAPER has proven to be a surprisingly stable playback system, considering the main playback project is a five-hour-long single session containing all pieces. However, due to scalability issues, we are looking into replacing REAPER with a custom playback solution.

The biggest challenge in operating a system like this is its inherent complexity. Problems are often difficult to debug due to the large number of interacting components, especially when they are difficult to reproduce or are related to driver-level issues. The audio drivers themselves pose an additional challenge, because both the MADI and Dante drivers lack mature Linux support. This further motivates our interest in AES67.

From an artistic perspective, we have found that, in some cases, it is preferable to provide artists with a consistent and pleasurable experience than to have a "realistic" playback system. A clear example of this is the Ambisonics encoding, in

[21] https://cmake.org/

[22] https://github.com/tu-studio/seamless-install-maintain

which all sounds are encoded onto a sphere around the listener. In the SeamLess system, virtual sources are placed in 3D space. Thus, for consistency, a distance emulation was added to the ambisonics encoder using a gain based on the distance from the center. A physically correct gain curve would follow $1/r$ for the sound pressure, where r is the distance. However, this would leads to a rapid volume decrease while also increasing volume significantly towards the center. Instead, we chose a curve that approaches unity towards the center and falls of linearly after exiting a near field around the center. Additionally, virtual ambisonic sources are encoded more omnidirectionally in the near field to improve the user experience.

In the process of consolidating the system we decided to unify and normalize the coordinate systems in each room. This way, all pieces could be played in each room. However, this required changing the coordinates in all configuration files and old pieces. Consequently, many Python and Lua scripts for REAPER had to be created to scale and rotate, in some cases also mirror the room. There are also some minor differences between JACK and Pipewire-jack, such as how PipeWire selects threads when calling the buffer size changed callback function.

5. CONCLUSIONS

We presented the SeamLess system, a modular, open-source spatial audio rendering platform built entirely on the Linux audio ecosystem. By distributing the rendering workload across multiple commodity servers and separating control from processing, SeamLess enables large-scale loudspeaker installations. Currently, it can drive up to 832 channels, while keeping the creative workflow accessible to artists through familiar DAW-based tools and OSC control.

The system's reliance on JACK and, more recently, PipeWire, shows that the Linux audio stack is mature enough to support demanding, high-channel-count spatial audio applications in production environments. The ongoing transition to PipeWire is particularly promising. Its native AES67 support and WirePlumber's scripting capabilities have the potential to considerably simplify networked audio setups. We hope that our experience with PipeWire at this scale will contribute useful feedback to the wider Linux audio community.

Operational experience at the Berlin Humboldt Forum, where the system has run continuously since May 2025, confirms that the combination of systemd services, Ansible-based deployment and strict version control provides the reliability required for permanent installations. Looking ahead, we plan to replace the proprietary REAPER-based playback engine with a custom, open-source solution. We also plan to further investigate the stability of PipeWire under sustained, high-channel-count operation, and explore additional rendering methods beyond WFS and Ambisonics. The fully open-source nature of the project invites adoption and contributions from other institutions operating comparable setups. We believe this distributed, Linux-based rendering can make spatial audio more accessible, both for large-scale venues and for smaller, budget-conscious installations.

6. ACKNOWLEDGMENTS

- Paul Schuladen and Nils Tonnätt, for their work on the first version of the SeamLess system.
- Wim Taymans and the PipeWire project team for their excellent work, quick responses, and fixes.
- The team at the IEM for the phenomenal plugin suite.

7. REFERENCES

[1] Frank Schultz, Nara Hahn, and Sascha Spors, "Wellenfeldsynthese," in *Handbuch der Audiotechnik*, Stefan Weinzierl, Ed., pp. 595–614. Springer, Berlin, Heidelberg, 2025.

[2] Hagen Wierstorf, Alexander Raake, and Matthias Geier, "Perception of Focused Sources in Wave Field Synthesis," *Journal of the Audio Engineering Society*, vol. 61, no. 1/2, pp. 5–16, 2013.

[3] Henrik von Coler, Paul Schuladen, and Nils Tonnätt, "SeamLess Integration of Spatial Sound Reproduction Methods," in *Proceedings of the 47th International Computer Music Conference*. 2021, International Computer Music Association.

[4] Alexander Lindau, Ricarda Kopal, Albrecht Wiedmann, and Stefan Weinzierl, "Räumliche Schallfeldsynthese für eine musikethnologische Ausstellung: Erfahrungen aus Produktion und Rezeption," in *Proceedings of the Inter-Noise 2016*. 2016, pp. 577–580, Deutsche Gesellschaft für Akustik e.V.

[5] Thibaut Carpentier, Markus Noisternig, and Olivier Warusfel, "Twenty Years of Ircam Spat: Looking Back, Looking Forward," in *41th International Computer Music Conference (ICMC)*. 2015, International Computer Music Association.

[6] Matthias Geier, Jens Ahrens, and Sascha Spors, "The SoundScape Renderer: A Unified Spatial Audio Reproduction Framework for Arbitrary Rendering Methods," in *124th AES Convention*. 2008, Audio Engineering Society.

[7] Matthias Geier, Torben Hohn, and Sascha Spors, "An Open-Source C++ Framework for Multithreaded Realtime Multichannel Audio Applications," in *Proceedings of the Linux Audio Conference*, 2012, pp. 183–188.

[8] Giso Grimm, Joanna Luberadzka, Tobias Herzke, Volker Hohmann, and Universitat Oldenburg, "Toolbox for acoustic scene creation and rendering (TASCAR): Render methods and research applications," in *Proceedings of the Linux Audio Conference*, 2015, pp. 9–12.

[9] Giso Grimm, Joanna Luberadzka, and Volker Hohmann, "A toolbox for rendering virtual acoustic environments in the context of audiology," *Acta Acustica united with Acustica*, vol. 105, no. 3, pp. 566–578, May 2019, arXiv:1804.11300 [cs].

[10] Daniel Rudrich, Frank Zotter, and M Frank, "Efficient Spatial Ambisonic Effects for Live Audio," in *29th VDT International Convention*. 2016, Verband Deutscher Tonmeister*innen e. V.

[11] Matthias Kronlachner, "Plug-in Suite for Mastering the Production and Playback in Surround Sound and Ambisonics," *Gold-Awarded Contribution to AES Student Design Competition*, 2014.

[12] Leo McCormack and Politis Archontis, "SPARTA & COMPASS: Real-Time Implementations of Linear and Parametric Spatial Audio Reproduction and Processing Methods," in *2019 AES International Conference on Immersive and Interactive Audio*. 2019, Audio Engineering Society.

[13] Marije Baalman and Daniel Plewe, "WONDER - a software interface for the application of Wave Field Synthesis in electronic music and interactive sound installations," in *Proceedings of the 30th International Computer Music Conference*. 2004, International Computer Music Association.

[14] Marije A.J. Baalman, *On Wave Field Synthesis and electro-acousti music, with a particular focus on the reproduction of arbitrarily shaped sound sources*, Ph.D. thesis, Technische Universität Berlin, 2008.

8. APPENDIX: CONFIG FILES

8.1. Config: Audio Matrix

```
tracks:
  - name: wfs
    modules:
      - gain:
          factor: 0
          osc_controllable: true
          osc_path: /source/send/wfs
  - name: hoa
    modules:
      - gain:
          factor: 0
          osc_controllable: true
          osc_path: /source/send/hoa
      - hoa_encoder:
          order: 3
          osc_controllable: true
          osc_path: /source/pos/aed
      - gain:
          factor: 0.492 # gain adjustment to
              match wfs
      - filter:
          type: HP
          freq: 100
      - delay: 8 # delay in ms to sync with the
          sub
  - name: sub
    modules:
      - gain:
          factor: 0
          osc_controllable: true
          osc_path: /source/send/sub
      - distance_gain:
          osc_path: /source/pos/dist
      - sum
      - gain: 0.1
      - filter:
          type: LP
          freq: 100
```

8.2. Config: Jack-Connection-Manager

```
# inputs to audio-matrix
- client: system:capture_
  n_channels: 32
  connections:
    - client: audio-matrix:input_
      start_index: 0

# Encoded Ambisonics to decoder
- client: system:capture_
  n_channels: 16
  start_index: 33
  connections:
    - client: AllRADecoder:in_
      start_index: 1

# Decoded Ambisonics to output
- client: AllRADecoder:out_
  n_channels: 45
  start_index: 1
  connections:
    - client: system:playback_
      start_index: 1

# Audio Matrix Ambisonics to decoder
- client: audio-matrix:hoa_
  n_channels: 16
  start_index: 0
  connections:
    - client: AllRADecoder:in_
      start_index: 1

# WFS
- client: audio-matrix:wfs_
  n_channels: 64
  start_index: 0
  connections:
    - client: twonder1:input
    - client: twonder2:input
    - client: twonder3:input
    - client: twonder4:input
- client: twonder1:speaker
  n_channels: 16
  connections:
    - client: system:playback_
- client: twonder2:speaker
  n_channels: 16
  connections:
    - client: system:playback_
      start_index: 17
- client: twonder3:speaker
  n_channels: 16
  connections:
    - client: system:playback_
      start_index: 33
- client: twonder4:speaker
  n_channels: 16
  connections:
    - client: system:playback_
      start_index: 49
```

8.3. Config: OSC-Kreuz

```
global:
  number_sources: 64
  render_units: ["ambi", "wfs"]
  room_scaling_factor: 6.046 # all incoming
      positon changes are scaled by this factor
  room_name: EN325
  room_polygon:
    - [3.023, 1.620, 1.4]
    - [3.023, -1.620, 1.4]
    - [1.620, -2.430, 1.4]
    - [-1.620, -2.430, 1.4]
    - [-3.023, -1.620, 1.4]
    - [-3.023, 1.620, 1.4]
    - [-1.620, 2.430, 1.4]
    - [1.620, 2.430, 1.4]

receivers:
  - type: audiomatrix
    hosts:
      - hostname: localhost
        port: 54321
    paths:
      - path: /source/send/hoa
        renderer: ambi
        type: gain
      - path: /source/pos/aed
        type: position
        format: aedrad
      - path: /source/send/lfe
        type: gain
        renderer: lfe
    updateintervall: 0
  - type: audiomatrix
    hosts:
      - hostname: wfs-receiver-1.local
        port: 54321
      - hostname: wfs-receiver-2.local
        port: 54321
    paths:
      - path: /source/send/wfs
        renderer: wfs
        type: gain
    updateintervall: 0
```